

An Introduction to Minio

If you need a minimal and scalable distributed object server, then your best bet is Minio. You can use it to store all your unstructured data such as photos, videos, container/VM images, log files and archives. Delve into this article to know more.



We usually tend to think of databases in terms of structured data like tables/columns or even attributes. However, modern applications generate lots of data that cannot be easily classified into structures. Instead, it exists in the form of blobs, objects or files. As an example, a blogging server may use a SQL database for posts and authors, but it also has various other media files like videos, images, sound files, etc. Now, imagine a scenario where you are responsible for storing away backup files or log files in a secure manner and making them accessible.

Minio is an Apache licensed open source distributed object server that can help you here, and do a lot more. Written in the Go language, it has a minimal design and the goal is to get you started quickly. It is a great option with which to introduce an Object Store into your application. Minio is designed to be compatible with Amazon S3, which makes it easy to have the same API interface and make it

work seamlessly on your local systems and the cloud.

So, let's get started with setting up Minio on your local machine and writing an application to integrate with it.

Running a Minio server

There are a couple of ways in which you can run Minio. You can either download the binaries for your operating system/architecture from the [Downloads](#) page or you can use the Minio Docker image, if you are using Docker. We will go with the former option.

Go ahead and download the Minio Server distribution that applies to you. In my case, I downloaded a Windows build. From the [Downloads](#) page, you can get both the server (*minio.exe*) and its command-line client (*mc.exe*). Ensure that the binaries are available in the PATH.

While starting the Minio server, the key thing is to provide it a PATH value that points to the directory where